GENERADORES

Generadores en Python - La base del Streaming

Yield

Uso del return vacío

Puntero a un Generador

Partes de un Generador

Motivos para usar un Generador

Defectos de un Generador

Consumidor (Tirar del flujo de datos del Generador)

Funcionamiento del bucle "for" por dentro

Resumen del ciclo de vida completo de un Generador

Enviar (send) información a un Generador mientras se itera

Comprensión de Generadores (Generator Comprehension)

Detectar el cierre (close) de un Generador

Tipos de Generadores

Tipos de Estructuras de Datos

Orden de importancia de las Estructuras de Datos

Generadores en Python - La base del Streaming

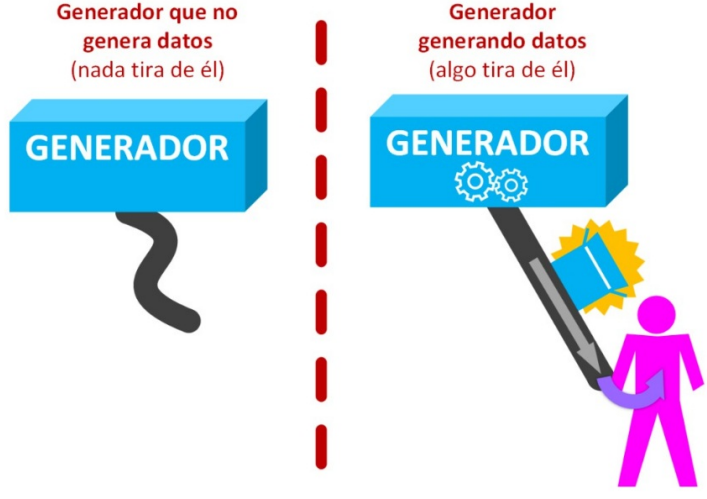
Un generador permite tener un flujo de datos en tiempo real (Streaming) y tener cada dato cuando necesite ser consumido (procesado). Los generadores garantizan un trabajo óptimo y equilibrio entre el proceso de datos (procesador) y la memoria principal (RAM).

Los generadores se usan igual que otras estructuras de datos de Python (list, dict o set). Podemos obtener los datos de un Generador mediante un bucle for:

```
for elemento in mi_generador:
        print(elemento)
```

Existe una DIFERENCIA IMPORTANTE: un generador no guarda los datos en ningún sitio, se genera cada dato cuando se solicita.

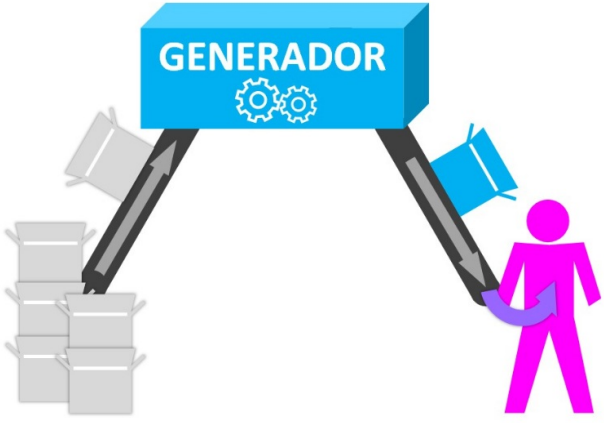
Podemos pensar que un generador es un objeto con una cinta transportadora que sale de dentro y para obtener un dato debemos tirar de la cinta transportadora.



Los datos de un Generador no existen hasta que son solicitados por un consumidor.

Generadores en Python - La base del Streaming

EJEMPLO: tenemos una pila de cajas blancas (datos sin procesar) y creamos un generador que pinte de azul cada caja (procesado de cada dato). Por tanto, cada vez que se tire de la cinta, una caja blanca entra por la cinta transportadora de entrada, será pintada por el Generador y se devuelve una caja pintada de azul (dato procesado) por la cinta transportadora de salida.

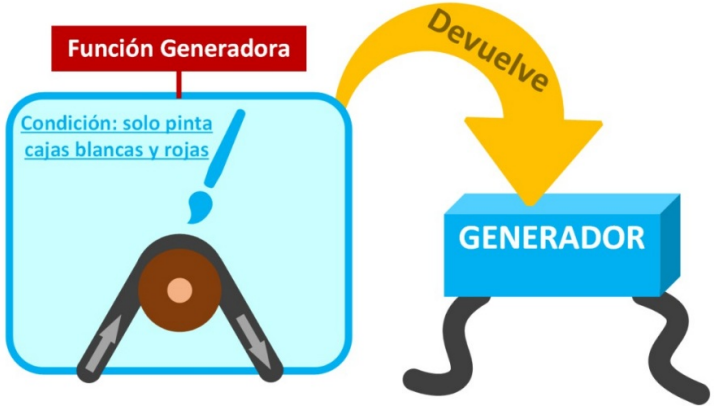


El generador se programa con una Función Generadora.

Vamos a complicarlo un poco: "la función generadora solo pinta de azul las cajas blancas y rojas, en caso de que sean de otro color se devuelven tal cual. Las cajas blancas se convierten a azul y las rojas a morado".

Una Función Generadora devuelve un Generador:

```
generador = funcion_generadora()
```



Generadores en Python - La base del Streaming

```
# FUNCIÓN GENERADORA. termina cuando se consume toda la ristra de cajas (con StopIteration)
def funcion_generadora(ristra_de_cajas):
    for caja in ristra_de_cajas:
        if caja == "BLANCO":
            yield "AZUL"
        elif caja == "ROJO":
            yield "MORADO"
        else:
            yield caja

ristra_de_cajas = ["BLANCO", "ROJO", "NEGRO"] # DATOS SIN PROCESAR

# para obtener el OBJETO GENERADOR se llama a la función generadora
generador = funcion_generadora(ristra_de_cajas)

# para obtener los DATOS PROCESADOS de un generador basta con iterarlo en un bucle
cajas_pintadas = []
for caja_pintada in generador:
    cajas_pintadas.append(caja_pintada)
    print(caja_pintada)
```

```
AZUL
MORADO
NEGRO
```

Yield

Significa "Producir/Generar". Una Función Generadora se crea cuando usa al menos una vez la palabra reservada yield dentro de una función de Python.

- La Función Generadora devuelve un Generador.
- Yield se pone en el momento que queremos devolver un dato procesado.

Ejemplo:

```
# termina cuando ejecuta la última instrucción con StopIteration
def funcion_generadora_prints():
    print("GENERADOR: Se va a generar un PRIMER dato")
    yield "valorGenerado1"

    print("GENERADOR: Se va a generar un SEGUNDO dato")
    yield "valorGenerado2"

    print("GENERADOR: Se va a generar un TERCER dato")
    yield "valorGenerado3"

    print("GENERADOR: Termina y lanzará automáticamente la excepción StopIteration")

generador2 = funcion_generadora_prints()

for x in generador2:
    print(x)
```

Yield y return NO son lo mismo.
En una función generadora solo se puede utilizar el return vacío (void) que si no se pone se ejecuta al llegar al final de la función.
Cuando se encuentra un return, en la función generadora se lanza automáticamente la excepción StopIteration y se termina la función.

```
GENERADOR: Se va a generar un PRIMER dato
valorGenerado1
GENERADOR: Se va a generar un SEGUNDO dato
valorGenerado2
GENERADOR: Se va a generar un TERCER dato
valorGenerado3
GENERADOR: Termina y lanzará automáticamente la excepción StopIteration
```

Yield

```
# una alternativa de ejecución sin el for
generador2 = funcion_generadora_prints()

try:
    primer_valor = next(generador)
    segundo_valor = next(generador)
    tercer_valor = next(generador)

    # Nuestro generador solo puede generar tres valores, por lo que llamar una cuarta vez a next lanza la excepción StopIteration
    cuarto_valor = next(generador)
except StopIteration:
    print("El Generador ya ha generado todos los valores posibles")
```

El Generador ya ha generado todos los valores posibles

Uso del return vacío

Típicamente se usa para evitar escribir condiciones extra para terminar el código. EJEMPLO: en la función generadora que pinta cajas, si llega cualquier color que no sea "BLANCO" o "ROJO" se termina inmediatamente el bucle y la función.

```
def funcion_generadora2(ristra_de_cajas):
    for caja in ristra_de_cajas:
        if caja == "BLANCO":
            yield "AZUL"
        elif caja == "ROJO":
            yield "MORADO"
        else:
            return # lanza StopIteration
```

```
generador3 = funcion_generadora2(ristra_de_cajas)

for caja in generador3:
    print(caja)
```

AZUL
MORADO

Puntero a un Generador

Para obtener un objeto Generador basta con llamar a la Función Generadora:

```
generador = funcion_generadora_prints()
```

Tenemos un objeto Generador pero NO ejecuta el código de la Función Generadora. Para ello, necesitamos algo que "tire" de la Función Generadora. Si hacemos:

```
print(generador)
```

Obtenemos el objeto generator que es un puntero a la función pero NO la invoca:

```
<generator object funcion_generadora at 0x00000123456789ABCDEF>
```

Para obtener los valores de un Generador basta con iterarlo en un bucle:

```
generador = funcion_generadora()
for elemento in generador:
        print(elemento)
```

```
def funcion_generadora(ristra_de_cajas):
        for caja in ristra_de_cajas:
                if caja == "BLANCO":
                        yield "AZUL"
                elif caja == "ROJO":
                        yield "MORADO"
                else:
                        yield caja
```

Partes de un Generador



Partes de un Generador

Función Generadora: tiene alguna instrucción yield. Al llamar a la Función Generadora devuelve un objeto tipo Generador.
Fuente (Entrada) de los datos: opcional. Puede que el Generador por sí mismo produzca los datos. En el ejemplo se pasa como parámetro "ristra de cajas". Puede ser cualquier iterador (incluso otro generador).
Lógica para el procesado de cada dato: en este caso la decisión de pintar solo las cajas "BLANCO" y "ROJO".
Bucle del que tirar: este bucle dentro de la Función Generadora ejecutará cada iteración cada vez que algo "tire". En este caso el que tira es el consumidor de datos. Este bucle es típico aunque hay casos en que no se usa (funcion_generadora_prints)
Producción de los datos (yield): cuando se llama a un "yield" dentro de la función generadora se devuelve un dato procesado y el control del flujo del programa fuera del Generador que se está iterando. En concreto al Consumidor.
Consumidor de los datos: es quien "tira" de los datos del objeto Generador. Cada vez que el consumidor "tira" de un dato del Generador, se pasa el control del flujo al Generador, y cuando el Generador produce un dato (llega a un yield), el consumidor retoma el control del flujo del programa para utilizar dicho dato. En el ejemplo es el for que itera el Generador:

```
ristra_de_cajas = ["BLANCO", "ROJO", "NEGRO"]
generador = funcion_generadora(ristra_de_cajas)

for caja_pintada in generador: # CONSUMIDOR
        print(caja_pintada)
```

Motivos para usar un Generador

El generador es una estructura de programación muy importante, más que list, dict, set, ficheros, etc.

Supongamos que resolvemos el código de pintar cajas con una función normal:

```
def pinta_de_azul(ristra_de_cajas): # solo modifica "BLANCA" y "ROJA"
        ristra_de_cajas_de_retorno = list()
        for caja in ristra_de_cajas:
                if caja == "BLANCA":
                        caja = "AZUL"
                elif caja == "ROJA":
                        caja = "MORADA"
                ristra_de_cajas_de_retorno.append(caja)
        return ristra_de_cajas_de_retorno
```

```
# La invocación es la misma
ristra_de_cajas = ["BLANCA", "ROJA", "NEGRA"]

ristra_de_cajas_pintadas = pinta_de_azul(ristra_de_cajas)
for caja in ristra_de_cajas_pintadas:
        print(caja)
```

El problema de no usar funciones generadoras es que el flujo del programa se bloquea dentro de esta función hasta que termina de procesar todas las cajas y devuelve la lista de retorno (CÓDIGO BLOQUEANTE). Por otro lado, se debe mantener en memoria el listado para devolverlo y procesarlo.

Estamos en la época del Big Data, imagina que pasa si el listado ristra_de_cajas tiene tantos datos que se tarda horas/días en procesar o, peor, el listado es infinito. De una forma u otra tardaríamos mucho en ver datos procesados o, en el caso de listado infinito, no los veríamos nunca.

Por otro lado, órdenes de magnitud de datos de Gigabytes, Terabytes o Petabytes implica tener una memoria RAM brutal.

Motivos para usar un Generador

Con la función generadora conseguimos:

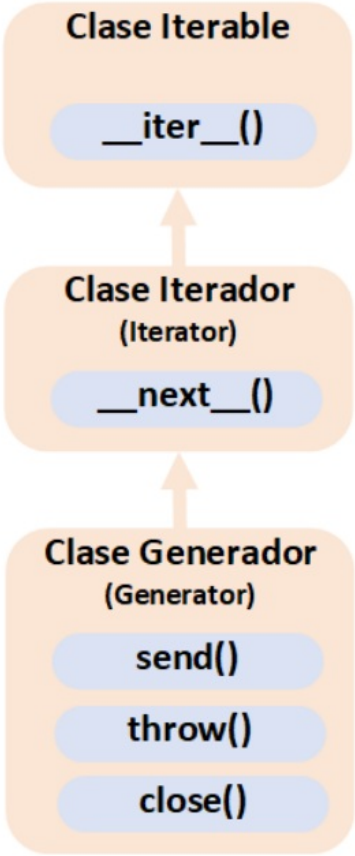
- Obtener cada uno de los datos tan pronto sea procesado y usarlo inmediantamente. Streaming/Flujo de datos.
- Consumir muy poca memoria RAM al descartar los datos ya usados. Optimización de la memoria RAM.
- Posibilidad de procesar infinitos datos. Flujo ininterrumpido de procesado de datos.
- Posibilidad de descartar datos que no sean interesantes. Filtrado de datos por Streaming.

Defectos de un Generador

- Un generador solo se puede iterar una única vez. Una vez agotado (StopIteration) se debe volver a crear con la Función Generadora.
- No se puede cambiar el orden de devolución de los datos. Con una lista se puede iterar en un sentido o en el contrario.
- No se puede ir a un dato en concreto: no hay índices.
- Procesado en tiempo de ejecución: lo que es bueno para la RAM es malo para el tiempo consumido por el procesador, ya que cada dato requiere ser procesado cada vez que se pide. Se puede arreglar haciendo que el generador sea tirado en un proceso aparte y que los datos se encolen en una cola (Buffer).
- Existen operaciones que no pueden utilizar Generadores: por ejemplo ordenar requiere de todos los datos para, dado un criterio de ordenación, poder ordenar.

Consumidor (Tirar del flujo de datos del Generador)

El objeto Generador es un Iterador: la clase "Generator" hereda de "Iterator" y "Iterator" de "Iterable".



Por tanto, para que el Generador produzca datos tenemos que utilizarlo como cualquier otro Iterador con un bucle "for". Un Iterador como Iterable puede utilizar "iter()" para iterar por todos sus elementos. También puede como Iterador usar "next()" para ir obteniendo elemento a elemento. Ejemplo:

```
def funcion_generadora_prints():
    print("GENERADOR: Se va a generar un PRIMER dato")
    yield "valorGenerado1"
    print("GENERADOR: Se va a generar un SEGUNDO dato")
    yield "valorGenerado2"
    print("GENERADOR: Se va a generar un TERCER dato")
    yield "valorGenerado3"
    print("GENERADOR: Se va a lanzar la excepción StopIteration")


generador2 = funcion_generadora_prints()
for valor_generado in generador2:
    print(f"Valor generado y obtenido desde el generador: {valor_generado}")
```

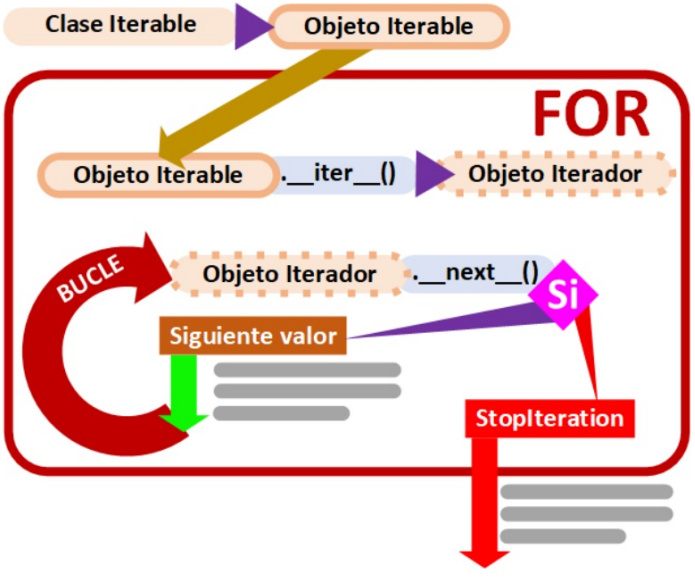
```
GENERADOR: Se va a generar un PRIMER dato
Valor generado y obtenido desde el generador: valorGenerado1
GENERADOR: Se va a generar un SEGUNDO dato
Valor generado y obtenido desde el generador: valorGenerado2
GENERADOR: Se va a generar un TERCER dato
Valor generado y obtenido desde el generador: valorGenerado3
GENERADOR: Se va a lanzar la excepción StopIteration
```

Consumidor (Tirar del flujo de datos del Generador)

El "for" tirará de un dato del Generador que hemos guardado en la variable valor_generado, lo obtiene y lo pinta en la consola con "print"; así uno a uno con todos los datos procesados. Cuando el Generador termine lanzará la excepción StopIteration que el bucle "for", que hace de Consumidor, capturará automáticamente (no hay que hacer nada con dicha excepción), el bucle "for" utiliza esta excepción para terminar de iterar y salir del bucle.

Funcionamiento del bucle "for" por dentro

```
for elemento in iterable:
        # Cuerpo del bucle for mientras no se detecte “StopIteration” o break
# Continuación del código, fuera del cuerpo del bucle for
```



El objeto iterable (list, dict, set, generator, etc.) es iterado como Iterador ("iter()") por el bucle "for". En cada iteración (llamada a "next()") se devuelve un elemento del iterador para ejecutar el código dentro de su cuerpo. Si se detecta la excepción "StopIteration", el bucle terminará y el código continuará ejecutándose tras el cuerpo del bucle "for".

Funcionamiento del bucle "for" por dentro

Si queremos recorrer los valores generados uno a uno podemos hacerlo con next(), pero necesitamos capturar y tratar la excepción StopIteration.

```
generador = funcion_generadora_prints()
try:
        primer_valor = next(generador)
        segundo_valor = next(generador)
        tercer_valor = next(generador)
        # Nuestro generador solo puede generar tres valores, por lo que llamar una cuarta vez a next lanza la excepción StopIteration
        cuarto_valor = next(generador)
except StopIteration:
        print("El Generador ya ha generado todos los valores posibles")
```

```
GENERADOR: Se va a generar un PRIMER dato
Valor generado y obtenido desde el generador: valorGenerado1
GENERADOR: Se va a generar un SEGUNDO dato
Valor generado y obtenido desde el generador: valorGenerado2
GENERADOR: Se va a generar un TERCER dato
Valor generado y obtenido desde el generador: valorGenerado3
GENERADOR: Se va a lanzar la excepción StopIteration
EXCEPCIÓN StopIteration: El Generador ya ha generado todos los valores posibles
```

Resumen del ciclo de vida completo de un Generador

1. Invocar la "función Generadora" para que cree el "Generador".
2. Se tira del flujo de datos del Generador (se itera) mediante un bucle "for" o sucesivas llamadas a "next()".
3. El Generador procesa un dato y se para en el primer "yield" que encuentra.
4. El "yield" devuelve ese dato procesado.
5. Se utiliza el dato procesado fuera de la "función Generadora".
6. Se devuelve el control a la "función Generadora" para que continúe hasta el siguiente "yield".
7. El Generador procesa un dato y se para en el siguiente "yield" que encuentra.
8. Repite desde el punto 4 hasta terminar todos los "yield" que puede alcanzar (si hay condiciones/filtrado puede que algún "yield" no se ejecute) dentro del cuerpo de la función o encontrar un "return".
9. Cuando se alcance el final de la "función Generadora" se lanza la excepción StopIteration automáticamente.
10. Si estaba recorriendo un bucle "for", se termina automáticamente al capturar la excepción StopIteration y continuará el código fuera del "for". Si se usa "next()", se controla la excepción con un "try ... catch StopIteration ..."
11. El "Generador" solo se puede recorrer una vez de principio a fin y será destruido por el "recolector de basura"

Enviar (send) información a un Generador mientras se itera

Se puede enviar información como parámetros:

```
mi_lista = [1, 2, 3]
mi_string = "un texto"
mi_booleano = True
Generador = funcion_generadora_parametros(mi_list, mi_string, mi_booleano)
```

Pero, además, se puede enviar información al Generador mientras un consumidor lo está iterando: método "send()".
Se suele usar para enviar señales ("flags") al generador para modificar su comportamiento.
"send()" es como "next()" que provoca llegar al siguiente "yield" por lo que hay que capturar la excepción StopIteration.
Ejemplo: generador que cuenta infinitamente y queremos poder cambiar su "paso", cuando nos de la gana, enviando un nuevo valor a sumar.

```
# utilizamos la retroalimentación del Generador (variable = yield ...)
# y protegemos por si se retroalimenta con un None
def funcion_generadora_contador_infinito(valor_a_sumar):
    acumulado = 0
    while True:
        acumulado += valor_a_sumar
        nuevo_valor_a_sumar = yield acumulado

        # Es importante comprobar el valor devuelto en el yield,
        # ya que si solo se tira sin llamar a send devolverá None
        if nuevo_valor_a_sumar is not None:
            valor_a_sumar = nuevo_valor_a_sumar
```

Aquí no se captura StopIteration ya que es un bucle infinito.

```
valor_de_inicio_a_sumar = 1
generador4 = funcion_generadora_contador_infinito(valor_de_inicio_a_sumar)

valor_generado = next(generador4)
print(f"ANTES   valor generado: {valor_generado}")
valor_generado = next(generador4)
print(f"ANTES   valor generado: {valor_generado}")

# Cuando queramos usamos send, que devolverá un valor al igual que next,
# ya que se envía el valor al generador y busca el siguiente yield
nuevo_valor_a_sumar_y_enviar = 10
valor_generado = generador4.send(nuevo_valor_a_sumar_y_enviar)
print(f"ENVIAR  valor generado: {valor_generado}")
valor_generado = next(generador4)
print(f"DESPUES valor generado: {valor_generado}")
valor_generado = next(generador4)
print(f"DESPUES valor generado: {valor_generado}")
```

```
ANTES   valor generado: 1
ANTES   valor generado: 2
ENVIAR  valor generado: 12
DESPUES valor generado: 22
DESPUES valor generado: 32
```

Ejemplo para Lexer y Parser

Lexer: produce tokens. Es decir, coge una frase de un lenguaje (ie. español, Python, Java, ...) realiza el análisis léxico y devuelve las unidades de valor (palabras, palabras reservadas, ...) y su tipo. Por ejemplo, si tenemos una frase en español "Juan come pan" -> "Juan" es sujeto, "come" es verbo, "pan" es "sujeto" (en realidad es "objeto directo", parte del "predicado", pero simplificamos).
Parser: reconoce la estructura del lenguaje; es decir, para una frase en español comprueba si las frases están bien formadas sintácticamente (estructura: sujeto + verbo + predicado). Parser("come Juan pan") -> incorrecto, Parser("Juan come pan") -> correcto.

Aquí usamos "send()" para alimentar el Lexer desde el Parser ya que hay tokens que no se pueden identificar sin su contexto. En español es fácil identificar los artículos (la, los, un, una, etc.) y las preposiciones (a, ante, bajo, cabe, con, etc.) con los listsdos completos. Pero los sujetos y verbos son infinitos (casi) y no tiene sentido tener listados. Por contexto podemos distinguir, un artículo siempre va delante de un sujeto o un verbo va siempre detrás de un sujeto, mientras no venga después de un artículo o preposición (casi seguro).

Si queremos hacer un Lexer sencillo de español, creamos un Generador al que llega un texto y lo separa por espacios. Tendremos los conjuntos de preposiciones y artículos que son sencillos de identificar pero el sujeto y verbo no. El Parser irá a parte (retroalimentado por el Parser variable = yield ...) y protegemos que no se retroalimente con None.

Ejemplo para Lexer y Parser

```
def funcion_generadora_lexer(texto):
    articulos = {"el", "los", "la", "las", "un", "unos", "una", "unas"}
    preposiciones = {
        "a", "ante", "bajo", "cabe","con", "contra", "de", "desde",
        "en", "entre", "hacia", "hasta", "para", "por", "según",
        "sin", "so", "sobre", "tras"
        }
    sujeto_o_verbo = "sujeto"
    palabras = texto.lower().split(" ")
    for palabra in palabras:
        if palabra in articulos:
            nuevo_sujeto_o_verbo = yield palabra, "artículo"
        elif palabra in preposiciones:
            nuevo_sujeto_o_verbo = yield palabra, "preposición"
        else:
            nuevo_sujeto_o_verbo = yield palabra, sujeto_o_verbo
        if nuevo_sujeto_o_verbo is not None:
            sujeto_o_verbo = nuevo_sujeto_o_verbo
```

```
La palabra 'juan' es de tipo 'sujeto'
La palabra 'come' es de tipo 'verbo'
La palabra 'pan' es de tipo 'sujeto'
La palabra 'de' es de tipo 'preposición'
La palabra 'la' es de tipo 'artículo'
La palabra 'panadería' es de tipo 'sujeto'
```

```
# Y ahora el Parser que lo simplifico recorriendo el Generador:
# Si viene un “artículo” -> lo que puede venir después es un “sujeto”
# Si viene un “sujeto” lo siguiente tendrá que ser un “verbo”, siempre que # no sea ni
“artículo” ni “preposición” (esto ya lo controla el Lexer)
# Si viene un “verbo” lo siguiente tendrá que ser “sujeto”

texto = "Juan come pan de la panadería"
generador = funcion_generadora_lexer(texto)
for palabra, tipo in generador:
        print(f"La palabra '{palabra}' es de tipo '{tipo}'")
        try:
                if tipo == "artículo":
                        palabra, tipo = generador.send("sujeto")
                        print(f"La palabra '{palabra}' es de tipo '{tipo}'")
                if tipo == "sujeto":
                        palabra, tipo = generador.send("verbo")
                        print(f"La palabra '{palabra}' es de tipo '{tipo}'")
                if tipo == "verbo":
                        palabra, tipo = generador.send("sujeto")
                        print(f"La palabra '{palabra}' es de tipo '{tipo}'")
        except StopIteration:
                pass
```

Comprensión de Generadores (Generator Comprehension)

Un generador se puede comprimir.

Los Generadores se delimitan con paréntesis “(” “)”.

Si queremos generar los múltiplos de 10 hasta un número muy grande:

```
generador = (num * 10 for num in range(1000000000))
```

Podremos probarlo como cualquier otro generador:

```
for numero in generador:
    print(numero)
```

Mostrará inmediatamente la salida:

0

10

20

30

...

OJO con la creación de estructuras no deseadas:

```
lista_gigante = [num*10 for num in range(1000000000)]
for numero in lista_gigante:
    print(numero)
```

Puede que tarde minutos en crear la lista antes de entrar al bucle o, si te quedas sin memoria, el SOP terminará matando el proceso.

Detectar el cierre (close) de un Generador

No suele hacer falta cerrar un Generador, pero si lo necesitamos podremos hacerlo con el método "close()".

```
generador.close()
```

Realmente no tenemos que llamar a "close" nosotros debido a que es llamado automáticamente por el "recolector de basura" (Garbage Collector) de Python cuando el Generador no va a ser utilizado más (porque se ha terminado de recorrer o ya no hay variables que apunten al Generador).

Por lo que, terminando de recorrer al Generador o eliminando el puntero de todas las variables será llamado "close" automáticamente; es decir, bastaría con llamar a "del" para eliminar el puntero y que el recolector de basura haga su trabajo:

```
del generador
```

El método "close" va a lanzar una excepción "GeneratorExit" dentro del Generador:

```
def funcion_generadora_print():
    try:
        print("GENERADOR: Se va a generar un PRIMER dato")
        yield "valorGenerado1"
        print("GENERADOR: Se va a generar un SEGUNDO dato")
        yield "valorGenerado2"
        print("GENERADOR: Se va a generar un TERCER dato")
        yield "valorGenerado3"
    except GeneratorExit:
        print("GENERADOR: Se ha capturado la excepción
        GeneratorExit")
```

```
generador = funcion_generadora_print()
print(next(generador))
print(next(generador))
print("Cerrando el generador")
del generador # Y el recolector de basura de Python
llama a generador.close()
```

```
GENERADOR: Se va a generar un PRIMER dato
valorGenerado1
GENERADOR: Se va a generar un SEGUNDO dato
valorGenerado2
Cerrando el generador
GENERADOR: Se ha capturado la excepción GeneratorExit
```

Detectar el cierre (close) de un Generador

Pero no es una buena práctica capturar la excepción "GeneratorExit" si lo que queremos es detectar la terminación del Generador, ya que si tuviéramos cualquier otro error dentro del Generador nunca entraría por la captura de la excepción "GeneratorExit" (por el "except GeneratorExit") cuando el "close" sea llamado (pues el Generador ya no estará disponible).

Si queremos detectar la terminación del Generador en todos los casos tendremos que utilizar "finally":

```
def funcion_generadora_print():
    try:
        print("GENERADOR: Se va a generar un PRIMER dato")
        yield "valorGenerado1"
        print("GENERADOR: Se va a generar un SEGUNDO dato")
        yield "valorGenerado2"
        print("GENERADOR: Se va a generar un TERCER dato")
        yield "valorGenerado3"
    finally:
        print("GENERADOR: Terminando y limpiando")
```

```
GENERADOR: Se va a generar un PRIMER dato
valorGenerado1
GENERADOR: Se va a generar un SEGUNDO dato
valorGenerado2
Cerrando el generador
GENERADOR: Terminando y limpiando
```

Por cierto, si agotáramos al Generador (hasta que lance StopIteration) también se limpiará automáticamente (y no tendremos ni que eliminar al Generador a mano con "del"):

```
for elemento in generador:
    print(elemento)
```

```
GENERADOR: Se va a generar un PRIMER dato
valorGenerado1
GENERADOR: Se va a generar un SEGUNDO dato
valorGenerado2
GENERADOR: Se va a generar un TERCERO dato
valorGenerado3
GENERADOR: Terminando y limpiando
```

Tipos de Generadores

Los Generadores se pueden clasificar en:

• Finitos: Devuelven un número determinado de datos; es decir, en algún momento deja de devolver datos y el flujo del programa sigue más allá del código de la función Generadora (por ejemplo, un Generador de números del 1 al 100000000000). Este tipo de Generadores se suele confundir con los listados (list) pero no es lo mismo.

• Infinitos: Devuelven datos eternamente; nunca van a dejar de devolver datos, por lo que no terminan y el flujo de programa se queda siempre en el mismo punto (aunque se podría forzar la terminación, por ejemplo, cuando me haya devuelto 100 datos ya no quiero más y fuerzo a terminar el bucle que lo recorre; si el bucle es "for" se puede forzar su detención con un "break"). Un problema de este tipo de Generadores es que se podría detener de manera indefinida esperando por el siguiente dato (por ejemplo, un hilo que consulte cada cinco segundos a un servidor en Internet de manera ininterrumpida y los datos consultados los devuelva a través de un Generador; esto es infinito, pero si el servidor se bloquea por lo que sea, el Generador también y el programa se quedará esperando por los datos del Generador).

Tipos de Generadores

• Indefinidos: Según los parámetros de entrada a la "función Generadora" que el Generador es finito o infinito. Existen dos tipos:

• Por definición Directa: Se le pasa algún parámetro (int, str, bool, etc.) que defina el tipo de Generador que se creará (finito o infinito). Por ejemplo, podríamos tener una "función Generadora" que devolviera palabras, y que esta "función Generadora" tenga un parámetro booleano que si es verdadero (true) devuelva palabras de un diccionario de la A hasta la Z una a una hasta agotarlas (Generador finito); y si el parámetro se pone falso (false) devuelva palabras del diccionario al azar eternamente (infinito).

• Por definición Indirecta (O Intermedios): Toman el flujo de datos (posiblemente datos provenientes de otro Generador, o de una fuente continua de datos como puede ser una consulta a Internet en tiempo real o "eterna"), lo procesan y lo devuelven, así hasta agotar el flujo de entrada (finito) o no (infinito); que a su vez el flujo de salida podría ser utilizado por otra "función Generadora" y por tanto por otro Generador. Este tipo de Generadores es el más interesante y los que más te van a ayudar en el tratamiento de los datos en flujo/Streaming, sobre todo muy útil a nivel profesional para procesar grandes cantidades de datos, al instante y en tiempo real. Este tipo de Generador también es muy útil para obtener los datos procesados desde otro hilo/proceso.

Tipos de Estructuras de Datos

En Python (y otros lenguajes) se pueden distinguir tres tipos de estructuras de datos:

• De memoria principal (memoria RAM): Listados (List), Diccionarios (Dict) o Conjuntos (Set). Lo bueno de estas estructuras es que son muy rápidas, pero tienen el inconveniente de usar la memoria principal que es escasa (menos que la secundaria, actualmente un ordenador puede tener unos "pocos" Gigabytes) y es cara. Un listado muy grande podría ser mayor que la memoria principal, lo que empujaría al SOP a matar al proceso que más se esté pasando con la memoria principal ¿Realmente existe algo tan grande? Big Data, es decir, intenta trabajar con datos de varios Terabytes en memoria principal (por ejemplo, diez años de datos sobre información meteorológica), no caben.

• De memoria secundaria (Disco duro): Ficheros (File) o Bases de datos (SQLite). De memoria secundaria se suele tener mucha (Un ordenador puede tener unos cuantos discos duros de varios Terabytes cada uno) ya que es muy "barata", pero es lentísima. Por ejemplo, podríamos buscar un dato en un fichero tremendamente grande, pero posiblemente nos eternicemos (hablamos de tardar minutos en completar una búsqueda si el fichero es muy grande, y si hay que realizar muchas búsquedas ya nos iríamos a horas o días).

• De procesado (apenas consume memoria RAM, pero sí mucho procesador): Generador (Generator). Al obtener cada uno de los datos en el momento que lo necesita procesar no consume apenas memoria principal. Lo que sí consumiría memoria sería el dato ya procesado, que hay que dejarlo en alguna parte (disco duro o memoria RAM). Una ventaja es poder utilizar cada dato a medida que se devuelve procesado y descartarlo inmediatamente para liberar memoria; por ejemplo, se podría enviar por Internet una vez procesado y así ya nos deshacemos de él, o bien se pinta por pantalla (se muestra en la consola) y ya no nos hace falta (liberamos la memoria), puede que lo tengamos que almacenar en un fichero y así no consumiría memoria principal (aunque sería lento, pero para eso existe el truco de tener otro hilo/proceso que consuma los datos procesados y que sea este otro hilo/proceso el encargado de guardarlos en fichero). El principal inconveniente que tiene el Generador es que los datos no existen hasta que se piden (por eso no se puede usar len(), porque no se sabe cuántos datos va a generar un Generador hasta que termine de procesar todos los datos, si es que es un generador finito).

Orden de importancia de las Estructuras de Datos

La estructura de datos tipo "Generador" es la más importante y la que más interesa entender para utilizarla desde ahora. Recuerda, el resto de las estructuras de datos también son importantes, pero el "Generador" es especial, primero porque es la más abstracta por lo que es ligeramente más compleja y que algunos comportamientos no se pueden sobreentender desde otras estructuras (por ejemplo, no se puede obtener la cantidad de sus elementos con "len()", pues en un Generador finito hasta que no se recorra entero no se sabrá cuantos elementos generará y si el Generador es infinito no tiene sentido).

Por estas razones, si tuviéramos que elegir una estructura de datos para el procesado de los datos tanto en tiempo real como para que el ordenador no se quede sin recursos con nuestro proceso (consuma lo mínimo y solo cuando tiene que consumir), tendríamos el siguiente orden de preferencia:

1. Generador: Los datos estarán en el momento que se necesiten, consumiendo la memoria RAM mínima. Deberíamos elegir esta estructura por encima del resto, salvo que no podamos (ver siguientes puntos y razones).
2. Diccionario: Si no podemos generar los datos porque ya los tenemos dados (no necesitamos generarlos, ya están pre-procesados o están cacheados), entonces un diccionario es una estructura ideal para realizar búsquedas inmediatas ¿Y si quiero recorrer los elementos del diccionario por el orden de inserción tendría que utilizar un listado? Mejor podemos utilizar OrderDict o en Python 3 ya se asegura el orden de inserción en dict, por lo que usar un diccionario en un bucle "for" ya recorrería en el orden de inserción.
3. Conjunto: Si no tenemos una clave que apunta a un valor y no necesitamos el orden de inserción, el conjunto debería ser la siguiente opción para tener en cuenta.
4. Listado: La verdad es que hay pocas excusas para usar listados. Una de las principales motivaciones de usar listados puede ser para ordenar valores, pues debemos tener todos los valores en memoria para ordenarlos; porque procesar los datos con otra estructura de datos sea menos eficiente que con un listado; o para recorrer varias veces los datos devueltos por un Generador (por si el procesado del Generador es pesado o único, como no poder realizar más consultas a un servidor por la gran carga de datos que supondría).

Orden de importancia de las Estructuras de Datos

5. Fichero (o base de datos): Si tenemos tantos datos que tenemos que guardarlos en ficheros para poder trabajar con ellos (debido a que la memoria RAM no tiene tanta capacidad); por ejemplo, para ordenar Terabytes de datos. O si queremos guardar el estado del proceso. Aunque se ha puesto el fichero en último lugar muchas veces se necesita utilizar por encima de cualquier otra estructura de datos por los motivos previamente indicados, pero al ser una estructura tan lenta no debería ser utilizado al menos que no quede otra.